

Query

Suppose a database consists of two tables. The first table is called employees, where each row contains an employee's ID, first name, last name, department, salary, and sales. The other table is called sales, where columns are ID of the employee and sale1, sale2, sale3, and sale4, which represent the sales made by the employee in the first, second, third, and fourth quarters of 2022.

ID	First Name	Last Name	Department	Salary	Select
1	Paul	Garrix	Corporate	3547	A
2	Astrid	Fox	Private	2845	A
3	Matthias	Johnson	Private	3009	A
4	Lucy	Patterson	Private	3547	A
5	Tom	Page	Corporate	5974	A
6	Claudia	Conte	Corporate	4714	A
7	Walter	Deer	Private	3547	A
8	Stephanie	Marx	Corporate	4123	A
9	Luca	Pavarotti	Private	4132	A
10	Victoria	Pollock	Corporate	4789	A

ID	Sale 1	Sale 2	Sale 3	Sale 4
8	3471.41	14789.25	3478.34	1254.39
9	12846.23	389.89	1548.67	2474.33
10	8715.55	8465.63	12223.34	8415.51
11	4897.98	5322.05	1845.66	15279.37
12	3977.65	2497.21	6944.35	7781.01
13	4485.36	8787.45	7648.90	

Now write down SQL query for all of these question

1. Show ID and salary for all employees.
2. Show the employee whose salary is more than 3800.
3. Show the employees in decreasing order of their salary.
4. Show the employees in decreasing order of their salary is more than 3800.
5. Show the employees whose first 6 month sales are more than 1500.
6. Show department-wise average salary and total number employee.
7. Show total sales of every employee with their ID and first name.
8. Show total sales of every employee with their ID and first name whose first 6 months' sales are more than 15000.
9. Write a procedure to compute the average sale in every quarter.
10. Write a trigger to update the sell class (from A to G) in the employees table if the total sale of an employee in the sales table is more than 40000.

Ans

Here are simplified, straightforward versions of each SQL statement/procedure/trigger. Adjust names/columns to match your database exactly.

1. Show ID and salary for all employees:

```
SELECT ID, Salary  
FROM employees;
```

2. Show employees whose salary is more than 3800:

```
SELECT *  
FROM employees  
WHERE Salary > 3800;
```

3. Show all employees in descending order by salary:

```
SELECT *  
FROM employees  
ORDER BY Salary DESC;
```

4. Show employees whose salary is more than 3800, in descending order by salary:

```
SELECT *  
FROM employees  
WHERE Salary > 3800  
ORDER BY Salary DESC;
```

5. Show employees whose first 6-month sales (Sale1 + Sale2) exceed 1500:

```
SELECT e.ID, e.First_Name, e.Last_Name
FROM employees e
JOIN sales s ON e.ID = s.ID
WHERE (s.Sale1 + s.Sale2) > 1500;
```

6. Show each department's average salary and total number of employees:

```
SELECT Department,
       AVG(Salary) AS Avg_Salary,
       COUNT(ID) AS Total_Employees
FROM employees
GROUP BY Department;
```

7. Show the total annual sales (Sale1 + Sale2 + Sale3 + Sale4) for each employee, with their ID and first name:

```
SELECT e.ID,
       e.First_Name,
       (s.Sale1 + s.Sale2 + s.Sale3 + s.Sale4) AS Total_Sales
FROM employees e
JOIN sales s ON e.ID = s.ID;
```

8. Show total annual sales for employees whose first 6-month sales (Sale1 + Sale2) exceed 15,000:

```
SELECT e.ID,
       e.First_Name,
```

```
(s.Sale1 + s.Sale2 + s.Sale3 + s.Sale4) AS Total_Sales  
FROM employees e  
JOIN sales s ON e.ID = s.ID  
WHERE (s.Sale1 + s.Sale2) > 15000;
```

9. Procedure to compute average sales in each quarter:

```
DELIMITER //  
CREATE PROCEDURE Compute_Avg_Sale()  
BEGIN  
    SELECT  
        AVG(Sale1) AS Avg_Sale_Q1,  
        AVG(Sale2) AS Avg_Sale_Q2,  
        AVG(Sale3) AS Avg_Sale_Q3,  
        AVG(Sale4) AS Avg_Sale_Q4  
    FROM sales;  
END //  
DELIMITER ;
```

10. Trigger to update the Select column (selling class) from 'A' to 'G' when total annual sales exceed 40,000:

```

DELIMITER //
CREATE TRIGGER Update_Sell_Class
AFTER INSERT OR UPDATE ON sales
FOR EACH ROW
BEGIN
    DECLARE total_sales DECIMAL(10,2);
    SET total_sales = NEW.Sale1 + NEW.Sale2 + NEW.Sale3 + NEW.Sale4;

    IF total_sales > 40000 THEN
        UPDATE employees
        SET `Select` = 'G'
        WHERE ID = NEW.ID;
    END IF;
END //
DELIMITER ;

```

Important:

- Replace table and column names if they differ in your database.
- If your column that tracks selling class is not actually named Select, change it to the correct name. If it is named Select, use backticks (`) around it as shown.

Relational Algebra in DBMS

Relational Algebra is a **procedural query language** used to retrieve data from relational databases. It provides **operators** to manipulate relations (tables) and produce new relations.

1. Basic Operators in Relational Algebra

1.1 Selection (σ) → Filtering Rows

- Selects rows that satisfy a given condition.

- **Notation:** $\sigma_{\text{condition}}(\text{Relation})$
- **Example:** Find students in "CSE" department.

$$\sigma_{\text{dept}='CSE'}(\text{Student})$$

1.2 Projection (π) → Selecting Columns

- Selects specific attributes (columns) from a relation.
- **Notation:** $\pi_{\text{attributes}}(\text{Relation})$
- **Example:** Get only names and departments of students.

$$\pi_{\text{Name, Dept}}(\text{Student})$$

1.3 Cartesian Product ($\times$) → Combining Relations

- Combines every row of one relation with every row of another.
- **Notation:** $R \times S$
- **Example:** Combine Students and Courses (no condition).

$$\text{Student} \times \text{Course}$$

- Produces **large results** (not efficient). Usually followed by **Selection (σ)**.

1.4 Union ($\cup$) → Combining Rows

- Returns **all unique tuples** present in either relation.
- **Conditions:**
 - Both relations must have **same attributes** and **data types**.
- **Example:** Get a list of all enrolled **CSE and EEE** students.

$$(\pi_{\text{ID, Name}}(\text{CSE}_{\text{students}})) \cup (\pi_{\text{ID, Name}}(\text{EEE}_{\text{students}}))$$

1.5 Set Difference ($-$) → Subtracting Rows

- Returns tuples that exist in **one relation but not in another**.

- **Example:** Find students **who are in CSE but not in EEE**.

$$(\pi_{ID, Name}(CSE_{students})) - (\pi_{ID, Name}(EEE_{students}))$$

1.6 Intersection ($\cap$) → Common Rows

- Returns tuples **present in both** relations.
- **Example:** Find students **enrolled in both CSE and EEE**.

$$(\pi_{ID, Name}(CSE_{students})) \cap (\pi_{ID, Name}(EEE_{students}))$$

1.7 Rename (ρ) → Naming Relations/Columns

- Used to rename attributes or relation names.

- **Notation:** $\rho_{newName}(Relation)$
- **Example:** Rename **Student** table to **Learner**.

$$\rho_{Learner}(Student)$$

2. Advanced Operators in Relational Algebra

2.1 Natural Join ($\bowtie$) → Matching Common Attributes

- Combines two relations **based on common attributes** (removes duplicates).
- **Notation:** $R \bowtie S$
- **Example:** Get **students with their departments** using dept_id.

$$Student \bowtie Department$$

2.2 Theta Join ($\bowtie_{condition}$) → Conditional Join

- Joins two relations **with a condition** (e.g., ID matching).

- **Example:** Find students enrolled in 'CSE' department.

$Student \bowtie_{Student.dept_id = Department.dept_id \wedge Department.name = 'CSE'} Department$

2.3 Division ($\div$) → Finding "All Matches"

- Used to find **entities that are related to all values** of another relation.
- **Example:** Find students who have enrolled in **all available courses**

$Enrolled \div Courses$

- Result: Students who have taken **every** course.

3. Relational Algebra Query Examples

Example 1: Find Names of Students Enrolled in "CSE"

$\pi_{Name}(\sigma_{dept='CSE'}(Student))$

Example 2: Find Students Who Have Taken "DBMS" Course

$\pi_{Student.Name}(Student \bowtie Enrolled \bowtie_{\sigma_{Course='DBMS'}}(Course))$

Example 3: Find Students Who Have Taken Both "DBMS" and "AI"

$(\pi_{Student.ID}(\sigma_{Course='DBMS'}(Enrolled))) \cap (\pi_{Student.ID}(\sigma_{Course='AI'}(Enrolled)))$

4. Relational Algebra vs SQL

Feature	Relational Algebra	SQL
Type	Procedural	Declarative
Operations	Uses operators like σ , π , $\bowtie$, $\cup$	Uses SELECT, FROM, WHERE, JOIN
Execution	Step-by-step operations	Direct query execution

Optimization	Manual optimization	Query optimizer handles it
---------------------	---------------------	----------------------------

5. Summary

Operator	Symbol	Function
Selection	σ	Filters rows
Projection	π	Selects columns
Cartesian Product	$\times$	Combines all rows
Union	$\cup$	Combines unique rows
Difference	$-$	Finds exclusive rows
Intersection	$\cap$	Finds common rows
Rename	ρ	Renames relation
Natural Join	$\bowtie$	Joins on common attributes
Theta Join	$\bowtie$ condition	Joins with condition
Division	$\div$	Matches "all" conditions

ER-Diagram:

Scenario: Library Management System

A library needs a system to manage its operations, including book lending, member records, and staff assignments. The system requirements are as follows:

1. The library contains many **Books**. Each book has attributes like Book_ID, Title, Author, Publisher, and Year_Published.
2. **Members** can **borrow** books from the library. Each member has a Member_ID, Name, Address, Phone, and Membership_Date.

3. Each book can be borrowed by one member at a time, but it can be borrowed multiple times over its lifecycle. This relationship involves attributes like Borrow_Date and Return_Date.
4. The library employs **Staff** who manage the book lending process. Staff members have Staff_ID, Name, Position, and Hire_Date.
5. **Categories** organize books. Each category has a Category_ID and Category_Name, and a book can belong to one or more categories.

Answer the following question for the above scenario:

1. Identify entities, relationships, and attributes
2. Draw an E-R diagram

Ans:

1. Entities, Relationships, and Attributes

Entities:

- **Books:**
 - Attributes: Book_ID, Title, Author, Publisher, Year_Published
- **Members:**
 - Attributes: Member_ID, Name, Address, Phone, Membership_Date
- **Staff:**
 - Attributes: Staff_ID, Name, Position, Hire_Date
- **Categories:**
 - Attributes: Category_ID, Category_Name

Relationships:

- **Borrows:** A relationship between Members and Books.
 - Attributes: Borrow_Date, Return_Date

Borrows (Members ↔ Books)
Attributes: Borrow_Date, Return_Date
Cardinality: Many-to-Many (M:N)
- **Manages:** A relationship between Staff and Books (related to lending process). **Manages (Staff ↔ Books - Lending Process)**
Cardinality: One-to-Many (1:M)

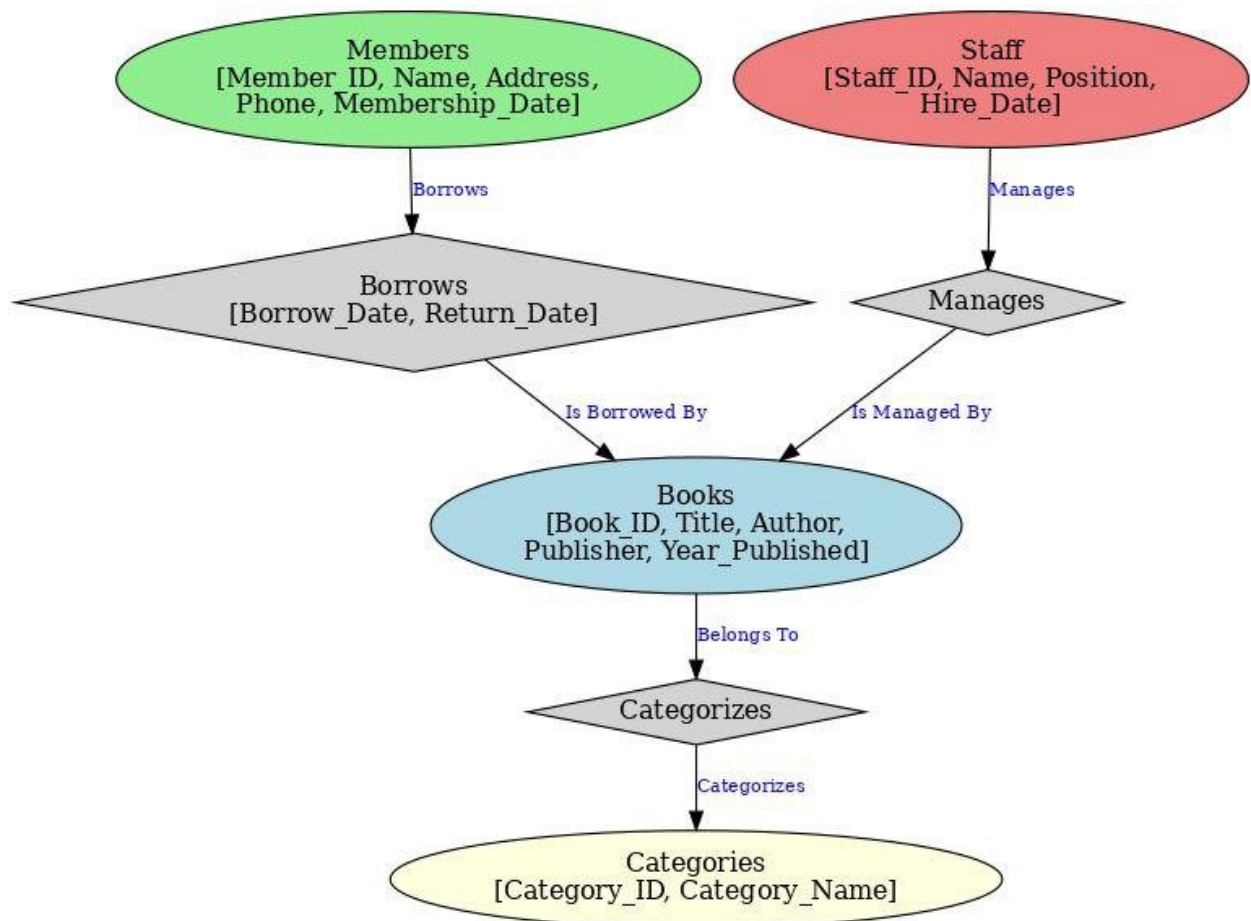
- **Categorizes:** A relationship between Books and Categories (a book can belong to one or more categories). **Categorizes (Books ↔ Categories)**
Cardinality: Many-to-Many (M:N)
-

2. E-R Diagram

The E-R Diagram would contain:

1. **Entities:** Books, Members, Staff, Categories
2. **Relationships:**
 - **Borrows:** Connects Members and Books.
 - **Manages:** Connects Staff and Books.
 - **Categorizes:** Connects Books and Categories.
3. **Attributes:**
 - Primary keys: Book_ID, Member_ID, Staff_ID, Category_ID
 - Other attributes are connected to their respective entities or relationships.

If you need the E-R diagram drawn, let me know, and I can provide a visual representation.



Procedure

Creating a Procedure to Get Employee Details by ID:

sql

```
CREATE PROCEDURE GetEmployee(IN emp_id INT)
BEGIN
    SELECT * FROM employees WHERE id = emp_id;
END;
```

Calling the Procedure:

sql

```
CALL GetEmployee(101);
```

Trigger

sql

```
CREATE TRIGGER trigger_name
{BEFORE | AFTER} {INSERT | UPDATE | DELETE}
ON table_name
FOR EACH ROW
BEGIN
    -- Trigger logic (SQL statements)
END;
```

